

Introduction

Every developer hits the same wall early on: you're tracking a handful of related values — a dog's name and age, a user's email and balance, a car's speed and fuel level — and you're doing it with separate variables. It works for one dog. It's annoying for two. By the time you need ten dogs or a hundred users, it's not a system anymore, it's a memory test you're guaranteed to fail.

This is exactly the problem classes exist to solve, and it's why "explain OOP" or "walk me through a class you designed" shows up constantly in coding interviews. Interviewers aren't testing whether you can recite the syntax — they're checking whether you understand *why* bundling data and behavior together is better than the alternative. If you can explain that reasoning clearly, you've demonstrated real engineering judgment, not memorization.

This lesson builds that understanding from the ground up: what breaks with loose variables, how a class fixes it, and how `__init__` and `self` work together to give every object built from a blueprint its own independent identity.

Problem Overview

Say you're building a program that needs to track dogs — each with a name and an age. The obvious first move is to create variables for each dog:

```
dog1_name = "Rex"
dog1_age = 3

dog2_name = "Milo"
dog2_age = 5
```

This technically works. But look closely at what's actually holding `dog1_name` and `dog1_age` together as *one dog's data*. Nothing. There's no structure enforcing that relationship — just a naming convention you have to remember and apply consistently, forever, by hand.

Now scale it up. Ten dogs means twenty variables, all named by a pattern you have to get right every single time: `dog3_name`, `dog3_age`, `dog4_name`, `dog4_age` ... One hundred users in a real application means two hundred loose values with no enforced connection between them. This isn't a system — it's a guessing game, and guessing games produce bugs: typos in variable names, mismatched updates, data that silently drifts out of sync.

The fix is to define a **blueprint** — a **class** — that describes what every dog *has* (its data) and what every dog *can do* (its behavior). Then each actual dog you create is an **object**: a real instance built from that blueprint, with its own values filled in.

Example

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says woof!")

rex = Dog("Rex", 3)
milo = Dog("Milo", 5)

print(rex.name, rex.age)    # Rex 3
print(milo.name, milo.age)  # Milo 5

rex.bark()    # Rex says woof!
milo.bark()   # Milo says woof!
```

Walk through what happened. `rex` and `milo` were both built from the exact same `Dog` class, but they hold completely independent data. Changing `rex.age` later would have zero effect on `milo.age`. There's no naming convention to maintain, no risk of accidentally mixing up whose age belongs to whom — the object itself keeps that boundary intact.

Intuition

Here's how an experienced engineer actually arrives at this solution, rather than being told to "use a class."

Start by asking: what data belongs together? A dog's name and age aren't independent facts floating in space — they describe *one thing*. The moment you notice yourself naming variables with a shared prefix (`dog1_` , `dog2_` , `user_a_`), that prefix is a signal. You're manually simulating a grouping that the language already has a built-in tool for.

The next question is: what can this thing *do*? A dog barks. A car accelerates. A user logs in. Behavior tied to a specific piece of data is a second signal that you're not looking at loose variables — you're looking at an *object*.

Once you see both signals — related data, plus actions on that data — the blueprint falls out naturally. You write the class once, describing the shape every object of that type will have. Then `__init__` becomes the answer to "how does an object get its starting values the moment it's created," and `self` becomes the answer to "how does a method know *which* object's data to touch." Neither is arbitrary syntax — each solves a problem you'd hit immediately if it didn't exist.

Brute Force Solution

Idea: Keep using separate, individually named variables for each entity, following a manual naming convention (`dog1_name` , `dog1_age` , `dog2_name` , ...).

Algorithm: 1. For each new dog, create two new variables following the `dogN_name` / `dogN_age` pattern. 2. Reference the correct pair by name whenever you need that dog's data. 3. Manually keep the naming consistent everywhere the data is used.

Advantages: - No new concepts required — works with only basic variable assignment. - Fine for exactly one or two fixed, never-repeated entities.

Disadvantages: - Doesn't scale — the number of variables grows linearly and unmanageably with each new entity. - No structural link between related variables; the relationship exists only in the programmer's head. - Extremely error-prone: typos, copy-paste mistakes, and mismatched indices are easy to introduce and hard to catch. - Can't be passed around as a single unit — you'd have to pass every variable individually into any function that needs them.

Time complexity: $O(1)$ to access a specific known variable, but $O(n)$ *variables* to create and maintain for n entities — and no way to loop over them programmatically. **Space complexity:** $O(n)$ for n entities, same as the class-based approach, but with much worse organization.

Optimal Solution

The optimal approach is to define a `class` once, and let it generate as many independent objects as needed.

Concept

```
class Dog:
    __init__(self, name,
age)
    self.name = name
```

What it means

Declares the blueprint — the template every dog object will follow.

Runs automatically the instant a new `Dog` is created; sets up its starting state.

Stores `name` as data belonging to *this specific* object, not a shared/global value.

Concept**What it means**`self` (in general)

A reference to "the object this method is currently running on."

`bark(self)`

A method — behavior attached to the object, able to read that object's own data.

The key mechanical detail is what `self` actually is. When you call `Dog("Rex", 3)`, Python creates a new, empty object in memory, then calls `__init__` on it automatically — passing that new object in as the first argument, which the method receives as `self`. So `self.name = name` literally means: "on this particular object being built right now, set its `name` attribute." That's why `rex.name` and `milo.name` never collide — they're written onto two entirely separate objects, even though `__init__` ran the identical line of code for both.

Methods work the same way. `rex.bark()` implicitly passes `rex` in as `self`, so inside `bark`, `self.name` resolves to `"Rex"`. Call `milo.bark()` and `self` becomes `milo` instead — same method code, different data, because `self` always points to whichever object is currently in use.

Python Code

```
class Dog:
    """Represents a single dog with a name and age."""

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says woof!")

    def have_birthday(self):
        self.age += 1

if __name__ == "__main__":
    rex = Dog("Rex", 3)
    milo = Dog("Milo", 5)

    rex.bark()
    milo.bark()

    rex.have_birthday()
    print(rex.age)    # 4
    print(milo.age)   # 5, unaffected by rex's change
```

Code Walkthrough

- `class Dog:` — declares the blueprint named `Dog`. No dog objects exist yet; this just defines the template.
- `def __init__(self, name, age):` — a special method Python calls automatically every time `Dog(...)` is invoked to create a new object.
- `self.name = name / self.age = age` — takes the values passed in and stores them as attributes *on the specific object being created*, accessible later as `object.name` and `object.age`.
- `def bark(self):` — a method, which is just a function defined inside a class. It automatically receives the calling object as `self`.
- `self.name` inside `bark` — reads the `name` attribute off whichever object called the method, without needing `name` passed in separately.
- `rex = Dog("Rex", 3)` — creates a new object, triggering `__init__` with `self` bound to this new object, `name="Rex"`, `age=3`.
- `rex.bark()` — Python translates this to `Dog.bark(rex)` under the hood, so `self` inside the method is `rex`.

Dry Run

Trace `rex = Dog("Rex", 3)` followed by `rex.bark()`:

1. Python allocates a new, empty `Dog` object in memory.
2. It calls `__init__(self=<new object>, name="Rex", age=3)`.
3. `self.name = "Rex"` writes `"Rex"` onto the new object's `name` attribute.
4. `self.age = 3` writes `3` onto the new object's `age` attribute.
5. The fully initialized object is assigned to the variable `rex`.
6. `rex.bark()` calls `bark(self=rex)`.
7. Inside `bark`, `self.name` looks up `rex`'s `name` attribute → `"Rex"`.
8. Output: `Rex` says woof!

Repeating this for `milo = Dog("Milo", 5)` creates a second, entirely separate object — its own memory, its own `name` and `age` — with zero interaction with `rex`'s data.

Complexity Analysis

- **Time:** Creating an object is $O(1)$ — `__init__` just assigns a fixed number of attributes regardless of how many other objects exist. Calling a method is also $O(1)$ to dispatch.

- **Space:** $O(n)$ for n objects, since each stores its own independent copy of its attributes — same asymptotic space as the loose-variable approach, but attributes are grouped and addressable through one reference (`rex`) instead of scattered across n separate variable names.
- **Why this is correct:** attribute storage is per-instance by design — Python allocates a distinct namespace for each object's `self`, which is precisely what guarantees `rex.age` and `milo.age` can never overwrite each other.

Alternative Solutions

For simple data-only cases with no behavior, Python's `dataclasses` module can reduce boilerplate:

```
from dataclasses import dataclass

@dataclass
class Dog:
    name: str
    age: int
```

This auto-generates `__init__` and a few other conveniences. It's a reasonable choice when a class is purely a data container. But the moment you need methods with real logic (like `bark` or `have_birthday`), a standard class is clearer and more explicit about what's happening — which matters most while you're still building the mental model of how objects work.

Edge Cases

- **No attributes passed:** `__init__(self)` with no other parameters is valid — you just get an empty-but-functional object.
- **Missing required arguments:** `Dog("Rex")` raises a `TypeError` if `age` has no default — Python enforces that `__init__`'s required parameters are supplied.
- **Mutable default values:** never write `def __init__(self, tags=[])` — the same list object gets shared across every instance that doesn't pass its own `tags`, corrupting state across objects. Use `None` and assign a new list inside `__init__` instead.
- **Very large numbers of objects:** works fine functionally, but for tens of thousands of lightweight objects, `__slots__` can reduce memory overhead — a later optimization, not a beginner concern.
- **Duplicate object values:** two `Dog` objects with identical `name` and `age` are still different objects in memory; `rex == another_rex` is `False` by default unless you define `__eq__`.

Common Mistakes

1. **Forgetting `self` as the first parameter** in instance methods — Python passes the calling object automatically, and omitting `self` breaks the method signature.
2. **Confusing the class with an object** — `Dog` is the blueprint; `rex` is an actual dog. You can't call `Dog.bark()` without an instance to act on.
3. **Using a class attribute when you meant an instance attribute** — defining `age = 0` directly under `class Dog:` (outside `__init__`) creates one shared value across all instances, not a per-object one.
4. **Mutable default arguments in `__init__`**, as covered above — a classic, hard-to-spot bug.
5. **Not calling `__init__` mentally when reasoning about object creation** — new learners often think attributes just "appear"; understanding that `__init__` runs automatically on every instantiation clears up a lot of confusion.
6. **Overloading a class with unrelated responsibilities** — if a `Dog` class starts managing file I/O or network calls, it's no longer modeling "a dog"; that's a design smell to watch for as programs grow.

Interview Questions

- What's the difference between a class and an object/instance?
- What does `self` actually refer to, and how does Python know what to pass as `self`?
- What happens if you forget to include `self` in a method definition?
- What's the difference between an instance attribute and a class attribute?
- Why is using a mutable default argument (like a list) in `__init__` dangerous?
- How would you make two `Dog` objects comparable with `==`?

Similar Problems

- **LeetCode 146 (LRU Cache)** — commonly implemented as a class with internal state (a hash map and a linked list) managed through `self`, a direct extension of this lesson's ideas.
- **LeetCode 380 (Insert Delete GetRandom O(1))** — requires designing a class that maintains internal data structures across multiple method calls.
- **LeetCode 355 (Design Twitter)** — a larger object-oriented design problem involving multiple interacting classes, a natural next step after mastering single-class design.
- **Any "Design a ____" LeetCode problem** — nearly all of them are testing whether you can model a real system as a class with clear attributes and methods, exactly the skill this lesson builds.

Key Takeaways

- Loose variables don't scale — past two or three entities, naming conventions break down and bugs creep in.
- A class is a blueprint describing what an object *has* (attributes) and what it *can do* (methods).
- `__init__` runs automatically on creation and sets up an object's starting state.
- `self` is how a method knows *which* object's data to read and modify — it's the mechanism that keeps every object's data independent.
- One class can produce unlimited objects, each with its own data but sharing the same behavior.

Watch the Video

For the full walkthrough with live coding and a step-by-step build of the `Dog` class, watch the video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series that builds real programming fluency one clear, practical lesson at a time — no fluff, no hand-waving, just the reasoning behind every concept so it actually sticks. Lesson 19 covers classes and objects; next up is inheritance, where one blueprint builds another.

Call To Action

If this clicked for you, the video walks through the same ideas visually — give it a watch and a like on YouTube, and subscribe so you don't miss Lesson 20 on inheritance. For the full written breakdown of every lesson, subscribe to the Substack newsletter. Got a question about `self` or `__init__`? Drop it in the comments — and if you know someone still fighting with `dog1_name` and `dog2_name`, send this their way.

Quick Review

Why replace many loose dog variables with a class?

Separate variables don't scale or stay linked; a class bundles related data and behavior into one reusable blueprint.

What does `__init__` do in a class?

Runs automatically when an object is created, setting up its starting instance attributes.

What is 'self' inside a method?

A reference to the specific object the method was called on, so it can read/change that object's own data.

Do two objects from the same class share attribute values?

No — each instance keeps its own separate copy of attributes set in `__init__`, even from one blueprint.

Common bug: forgetting 'self' as first param in a method?

Causes `TypeError` or wrong data access, since Python won't auto-bind the instance without it.

Class vs instance: what's the relationship?

Class is the blueprint (defines structure/methods); instance is one concrete object built from it with its own data.

Python Lesson 19: Object-Oriented Programming: Classes and Objects (Defining classes, `__init__`, instance attributes, and methods) — free Python cheat sheet